

Mini-Issue LAB 3: Redis 缓存与异步通知

一、LAB 目标

在 LAB1、LAB2 的基础上，为 Mini-Issue 系统加入两个后端常见能力：

1. Redis 缓存
2. 异步 worker

完成后，系统从：

HTTP API + MySQL CRUD

升级为：

HTTP API + MySQL + Redis Cache + Async Worker

本 LAB 的重点不是新增很多接口，而是学习：

什么时候查数据库
什么时候查缓存
什么时候删缓存
什么时候异步处理

二、需要提前学习的技术栈

1. Redis 基础

至少需要掌握：

SET
GET
DEL
EXPIRE
TTL
LPUSH
BRPOP

本 LAB 会用到两类 Redis 能力：

String: 做缓存
List: 做简单任务队列

2. Redis 缓存思想

需要理解:

缓存命中
缓存未命中
缓存过期时间 TTL
缓存删除
缓存一致性

重点提前理解:

Cache Aside Pattern

也就是:

读数据:
先查缓存
缓存没有再查数据库
查到数据库后写回缓存

写数据:
先改数据库
再删除缓存

3. Go Redis 客户端

建议使用:

github.com/redis/go-redis/v9

需要提前了解:

```
redis.NewClient  
client.Get  
client.Set  
client.Del
```

```
client.LPush  
client.BRPop
```

注意：

```
go-redis 大多数操作都需要 context.Context
```

4. Go context

至少理解：

```
context.Background()  
context.WithTimeout()
```

本 LAB 中：

```
Redis 操作都需要传 context
```

5. JSON 序列化

需要掌握：

```
json.Marshal  
json.Unmarshal
```

原因：

```
Redis 只能存字符串/字节  
Issue 结构体需要转成 JSON 后再缓存
```

6. Goroutine

需要掌握：

```
go func() {}  
for 循环后台任务
```

本 LAB 的 worker 会作为后台 goroutine 启动。

三、本 LAB 新增能力概览

1. Issue 详情缓存

接口：

```
GET /issues/:id
```

加入缓存。

流程：

```
先查 Redis
↓
命中则直接返回
↓
未命中则查 MySQL
↓
查完后写入 Redis
↓
返回结果
```

2. Issue 列表缓存

接口：

```
GET /issues?page=1&page_size=10
```

加入缓存。

流程：

```
先查 Redis
↓
命中则直接返回分页结果
↓
未命中则查 MySQL
↓
查完后写入 Redis
↓
返回结果
```

3. 缓存失效

以下操作后必须删除相关缓存：

```
POST /issues
PATCH /issues/:id
DELETE /issues/:id
```

原因：

```
MySQL 数据变了
Redis 里的旧数据不能继续用
```

4. 异步通知 worker

创建 issue 后，不在 HTTP 请求中直接处理通知，而是把通知任务推入 Redis 队列。

流程：

```
用户创建 issue
↓
MySQL 创建成功
↓
推送通知任务到 Redis List
↓
HTTP 请求立即返回
↓
后台 worker 慢慢消费任务
```

通知不用真的发邮件，本 LAB 只需要模拟：

```
打印日志
```

四、新增目录与文件

在现有项目基础上新增：

```
pkg/cache/
└── redis.go
```

```
internal/cache/  
└── issue_cache.go  
  
internal/worker/  
└── notification_worker.go  
  
internal/model/  
└── notification.go
```

如果你之前没有日志封装，也可以先直接使用：

```
log.Println
```

五、Redis 连接模块

文件：

```
pkg/cache/redis.go
```

目标

封装 Redis 客户端初始化。

需要实现的函数

```
NewRedisClient()
```

职责

1. 连接 Redis
2. 设置地址
3. 设置密码，如果没有密码则为空
4. 设置 DB 编号
5. Ping Redis，确认连接可用
6. 返回 Redis client

返回值建议

```
*redis.Client  
error
```

注意事项

Redis 地址建议先写死：

```
127.0.0.1:6379
```

后面 LAB 再改配置文件。

六、缓存 Key 设计

文件：

```
internal/cache/issue_cache.go
```

本文件只负责：

```
Issue 相关缓存操作
```

1. Issue 详情缓存 Key

格式：

```
issue:detail:{issue_id}
```

例子：

```
issue:detail:12
```

2. Issue 列表缓存 Key

格式：

```
issue:list:page:{page}:size:{page_size}
```

例子：

```
issue:list:page:1:size:10
```

3. 为什么列表缓存 Key 要带分页参数

因为：

```
?page=1&page_size=10
```

和：

```
?page=2&page_size=10
```

不是同一份数据。

所以缓存 key 必须区分。

七、IssueCache 设计

文件：

```
internal/cache/issue_cache.go
```

IssueCache 结构体

保存：

```
redisClient
```

类型：

```
*redis.Client
```

NewIssueCache

作用：

创建 IssueCache 对象

参数：

redisClient

返回：

*IssueCache

八、Issue 详情缓存函数

文件：

internal/cache/issue_cache.go

GetIssueDetail

作用：

从 Redis 查询 issue 详情缓存

参数：

issueID

返回建议：

(*model.Issue, error)

行为要求

1. 缓存命中

Redis 中存在 key。

需要：

1. 读取 JSON 字符串
2. 反序列化成 Issue
3. 返回 Issue

返回：

issue, nil

2. 缓存未命中

Redis 返回 nil。

返回：

nil, nil

含义：

没有缓存，但不是错误

3. Redis 错误

返回：

nil, err

含义：

Redis 查询失败

SetIssueDetail

作用：

把 issue 详情写入 Redis

参数：

issue
ttl

返回：

error

要求：

1. 将 Issue 序列化为 JSON
2. 写入 Redis
3. 设置过期时间

TTL 建议：

5 分钟

DeleteIssueDetail

作用：

删除某个 issue 的详情缓存

参数：

issueID

返回：

error

调用场景：

```
更新 issue 后  
删除 issue 后
```

九、Issue 列表缓存函数

文件：

```
internal/cache/issue_cache.go
```

GetIssueList

作用：

```
从 Redis 查询分页列表缓存
```

参数：

```
page  
pageSize
```

返回建议：

```
(*model.IssueListResponse, error)
```

SetIssueList

作用：

```
把分页列表写入 Redis
```

参数：

```
page  
pageSize
```

IssueListResponse
ttl

返回:

error

TTL 建议:

1 分钟

原因:

列表变化较频繁, TTL 不宜过长

DeleteIssueList

作用:

删除列表缓存

问题:

列表缓存可能有很多页

例如:

```
issue:list:page:1:size:10  
issue:list:page:2:size:10  
issue:list:page:1:size:20
```

本 LAB 简化处理:

只删除 page=1,page_size=10 的列表缓存

后续进阶再处理批量删除或 key pattern 删除。

十、Service 层改造

文件：

```
internal/service/issue_service.go
```

你需要修改已有的 IssueService。

IssueService 新增依赖

原来：

```
issueDAO
```

现在增加：

```
issueCache  
notificationProducer
```

也就是：

```
IssueService  
├── issueDAO  
├── issueCache  
└── notificationProducer
```

十一、GetIssueByID 改造

原 LAB2 流程：

```
查 MySQL  
↓  
返回
```

LAB3 改成：

```
查 Redis  
↓  
缓存命中：返回
```

↓
缓存未命中：查 MySQL
↓
MySQL 查到后：写入 Redis
↓
返回

需要注意

如果 Redis 出错，不要直接让接口失败。

推荐策略：

Redis 错误只记录日志
继续查 MySQL

原因：

缓存只是加速层
数据库才是主数据源

十二、ListIssues 改造

原 LAB2 流程：

查 MySQL list
查 MySQL count
返回

LAB3 改成：

查 Redis list cache
↓
命中：返回
↓
未命中：查 MySQL
↓
写 Redis
↓
返回

注意事项

列表缓存 TTL 比详情缓存短。

建议：

详情缓存：5 分钟
列表缓存：1 分钟

十三、CreateIssue 改造

原 LAB2 流程：

校验参数
↓
写 MySQL
↓
返回

LAB3 改成：

校验参数
↓
写 MySQL
↓
删除列表缓存
↓
推送通知任务
↓
返回

为什么创建后要删除列表缓存

因为新 issue 出现后：

GET /issues?page=1&page_size=10

应该能看到新数据。

如果不删缓存，用户可能看到旧列表。

十四、UpdateIssue 改造

原 LAB2 流程：

```
查 issue
权限判断
更新 MySQL
返回
```

LAB3 改成：

```
查 issue
权限判断
更新 MySQL
删除详情缓存
删除列表缓存
返回
```

删除哪些缓存

至少删除：

```
issue:detail:{issue_id}
issue:list:page:1:size:10
```

十五、DeleteIssue 改造

原 LAB2 流程：

```
查 issue
权限判断
删除 MySQL
返回
```

LAB3 改成：

```
查 issue
权限判断
删除 MySQL
删除详情缓存
```

删除列表缓存
返回

十六、异步通知模型

文件：

internal/model/notification.go

NotificationTask

字段建议：

字段	说明
Type	通知类型
UserID	用户 ID
IssueID	issue ID
Title	issue 标题
CreatedAt	创建时间

Type 建议

先只支持：

ISSUE_CREATED

后续可以加：

ISSUE_UPDATED
ISSUE_DELETED
COMMENT_CREATED

十七、通知队列设计

使用 Redis List。

队列 Key

notification:queue

生产者

创建 issue 后执行：

把 NotificationTask 序列化成 JSON
LPUSH notification:queue task

消费者

worker 中执行：

BRPOP notification:queue 0

含义：

阻塞等待任务

没有任务时不会空转。

十八、NotificationProducer 设计

可以放在：

internal/worker/notification_worker.go

或者单独拆成：

internal/worker/notification_producer.go

PushNotificationTask

作用：

把通知任务推入 Redis 队列

参数：

NotificationTask

返回：

error

流程：

1. JSON 序列化
2. LPUSH 到 notification:queue

十九、NotificationWorker 设计

文件：

internal/worker/notification_worker.go

NotificationWorker 结构体

保存：

redisClient

NewNotificationWorker

作用：

创建 worker

参数:

redisClient

返回:

*NotificationWorker

Start

作用:

启动 worker 循环

要求:

1. 死循环
2. BRPOP notification:queue
3. 解析任务 JSON
4. 根据任务类型处理
5. 打印日志模拟通知

Worker 日志示例

创建 issue 后, worker 可以打印:

send notification: user=1 issue=10 title=xxx

二十、main.go 改造

文件:

cmd/main.go

启动流程从：

```
初始化 MySQL
创建 DAO
创建 Service
创建 Controller
创建 Router
启动 HTTP 服务
```

改成：

```
初始化 MySQL
初始化 Redis
创建 DAO
创建 Cache
创建 Worker/Producer
创建 Service
创建 Controller
创建 Router
启动 Worker goroutine
启动 HTTP 服务
```

注意

worker 应该在 HTTP server 启动前启动：

```
go notificationWorker.Start()
```

不要阻塞 main goroutine。

二十一、错误处理策略

本 LAB 要求你区分两类错误。

1. MySQL 错误

MySQL 是主存储。

如果 MySQL 失败：

接口应该失败

例如：

创建 issue 写 MySQL 失败

应该返回错误。

2. Redis 缓存错误

Redis 在缓存场景下是加速层。

如果 Redis GET 失败：

记录日志
继续查 MySQL

如果 Redis SET 失败：

记录日志
接口仍然可以成功

如果 Redis DEL 失败：

记录日志
接口仍然可以成功

3. Redis 队列错误

如果创建 issue 后推送通知任务失败：

记录日志
接口仍然可以成功

原因：

通知不是主流程
创建 issue 成功才是主流程

二十二、缓存一致性要求

本 LAB 使用最简单可靠的策略：

写数据库成功后，删除缓存

不要使用：

写数据库后直接更新缓存

原因：

更新缓存容易漏字段
也容易和分页列表产生不一致

必须删除缓存的场景

操作	删除详情缓存	删除列表缓存
CreateIssue	不需要	需要
UpdateIssue	需要	需要
DeleteIssue	需要	需要

二十三、验收标准

完成后必须通过以下测试。

1. Redis 连接成功

启动服务时：

Redis ping 成功

2. Issue 详情缓存命中

测试流程：

1. 创建 issue
2. 第一次 GET /issues/:id
3. 查看 Redis, 应该出现 issue:detail:{id}
4. 第二次 GET /issues/:id
5. 应该从缓存返回

你可以通过日志判断：

```
cache hit issue detail
cache miss issue detail
```

3. Issue 列表缓存命中

测试流程：

1. GET /issues?page=1&page_size=10
2. Redis 出现 issue:list:page:1:size:10
3. 再次 GET 同样参数
4. 应该从缓存返回

4. 创建 issue 后列表缓存失效

测试流程：

1. GET /issues?page=1&page_size=10, 生成缓存
2. POST /issues 创建新 issue
3. 检查列表缓存被删除
4. 再次 GET /issues?page=1&page_size=10, 重新查 MySQL 并写入缓存

5. 更新 issue 后详情缓存失效

测试流程：

1. GET /issues/:id, 生成详情缓存
2. PATCH /issues/:id
3. 检查 issue:detail:{id} 被删除
4. 再次 GET /issues/:id, 返回更新后的内容

6. 删除 issue 后缓存失效

测试流程：

1. GET /issues/:id, 生成详情缓存
2. DELETE /issues/:id
3. 检查 issue:detail:{id} 被删除
4. 再次 GET /issues/:id 返回 404

7. 创建 issue 后推送通知任务

测试流程：

1. 启动 worker
2. POST /issues
3. worker 打印通知日志

8. HTTP 请求不被 worker 阻塞

创建 issue 接口不应该等待通知处理完成。

要求：

通知失败不影响 issue 创建成功

二十四、建议实现顺序

严格按照这个顺序做。

Step 1: 先跑通 Redis 连接

只做：

pkg/cache/redis.go

验收：

服务启动时 Redis Ping 成功

Step 2: 实现 Issue 详情缓存

只改:

```
internal/cache/issue_cache.go  
internal/service/issue_service.go
```

先只缓存:

```
GET /issues/:id
```

不要急着做列表缓存。

Step 3: 实现详情缓存失效

改:

```
UpdateIssue  
DeleteIssue
```

要求:

```
更新/删除后删除 issue:detail:{id}
```

Step 4: 实现列表缓存

缓存:

```
GET /issues?page=1&page_size=10
```

Step 5: 实现列表缓存失效

改:

```
CreateIssue  
UpdateIssue  
DeleteIssue
```

要求：

这些操作后删除列表缓存

Step 6: 实现 NotificationTask

新增：

```
internal/model/notification.go
```

只定义任务结构。

Step 7: 实现 Producer

创建 issue 成功后：

```
LPUSH notification:queue
```

Step 8: 实现 Worker

后台：

```
BRPOP notification:queue
```

收到任务后打印日志。

Step 9: main.go 组装

把 Redis、Cache、Worker 注入到 Service。

二十五、你需要重点思考的问题

完成 LAB3 后，你应该能回答：

1. 为什么 Redis 挂了，接口还能正常工作？

因为：

缓存不是主数据源
MySQL 才是主数据源

2. 为什么更新数据库后要删除缓存？

因为：

旧缓存已经不可信

3. 为什么不直接更新缓存？

因为：

缓存可能有多个 key
分页缓存尤其复杂
删除比更新更安全

4. 为什么通知要异步？

因为：

通知不是创建 issue 的必要条件

主流程应该尽快返回。

5. Redis List 能不能当真正消息队列？

本 LAB 可以。

但生产中要考虑：

- 任务丢失
- 重试
- 死信队列
- 消费确认
- 幂等性

这些会在后续 LAB 再处理。

二十六、本 LAB 不要求做的事

本 LAB 暂时不做：

- Redis 分布式锁
- 缓存穿透防护
- 缓存雪崩防护
- 缓存击穿防护
- 消息重试
- 死信队列
- 真正邮件发送
- 批量删除所有列表缓存
- Docker Compose
- 配置文件系统

这些以后再加。

本 LAB 只要求：

- Redis 缓存
- 缓存失效
- Redis List 异步 worker

二十七、最终效果

完成 LAB3 后，你的系统具备：

- 用户系统
- Issue CRUD
- 资源权限控制
- Redis 详情缓存
- Redis 列表缓存

缓存失效
Redis 队列
后台 worker
异步通知

这时你的项目已经不是普通 CRUD，而是一个具备真实后端系统雏形的小项目。